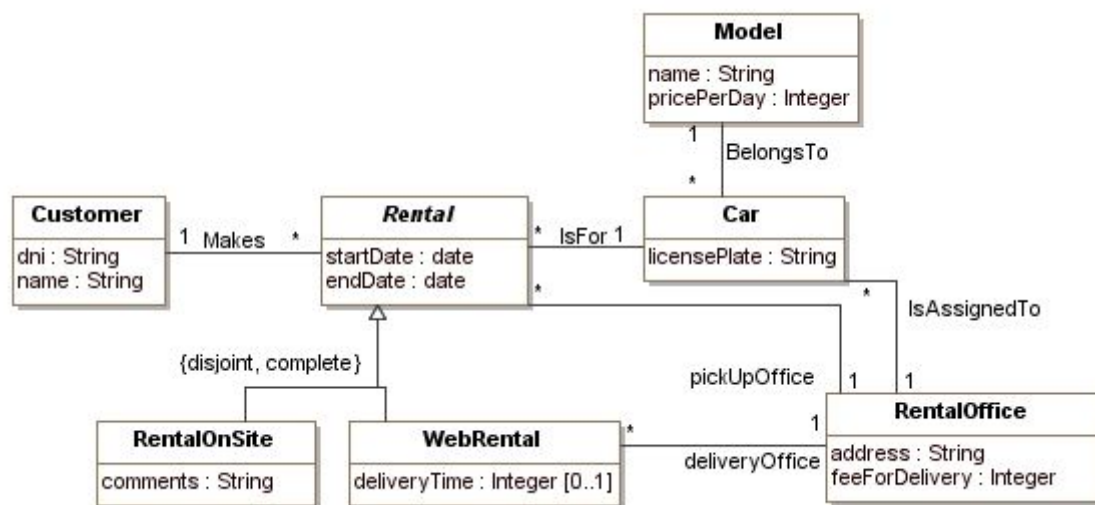


Análisis y Diseño con Patrones

Práctica 2: Patrones de Diseño y de Asignación de Responsabilidades

La empresa que nos contrató en la Práctica 1 para desarrollar un software para gestionar los alquileres de coches que hacen sus clientes quiere que añadamos nuevas funcionalidades al software. A continuación disponéis del diagrama de clases de la Práctica 1:

Diagrama de clases



Claves de las clases:

- *Customer*: `dni`
- *Car*: `licensePlate`
- *RentalOffice*: `address`
- *Model*: `name`
- *Rental*: `dni (Customer) + startDate`

Restricciones de integridad:

- Un cliente no puede tener alquileres solapados.
- La fecha de inicio de un alquiler tiene que ser más pequeña que la fecha final del alquiler.
- La oficina de recogida de un coche de alquiler tiene que ser la misma que la oficina donde está asignado el coche de alquiler.

- Si la oficina de recogida y de entrega de un alquiler web son diferentes, la hora de entrega del coche de alquiler tiene que ser anterior a las 13 horas.

Ejercicio 1 (15%)

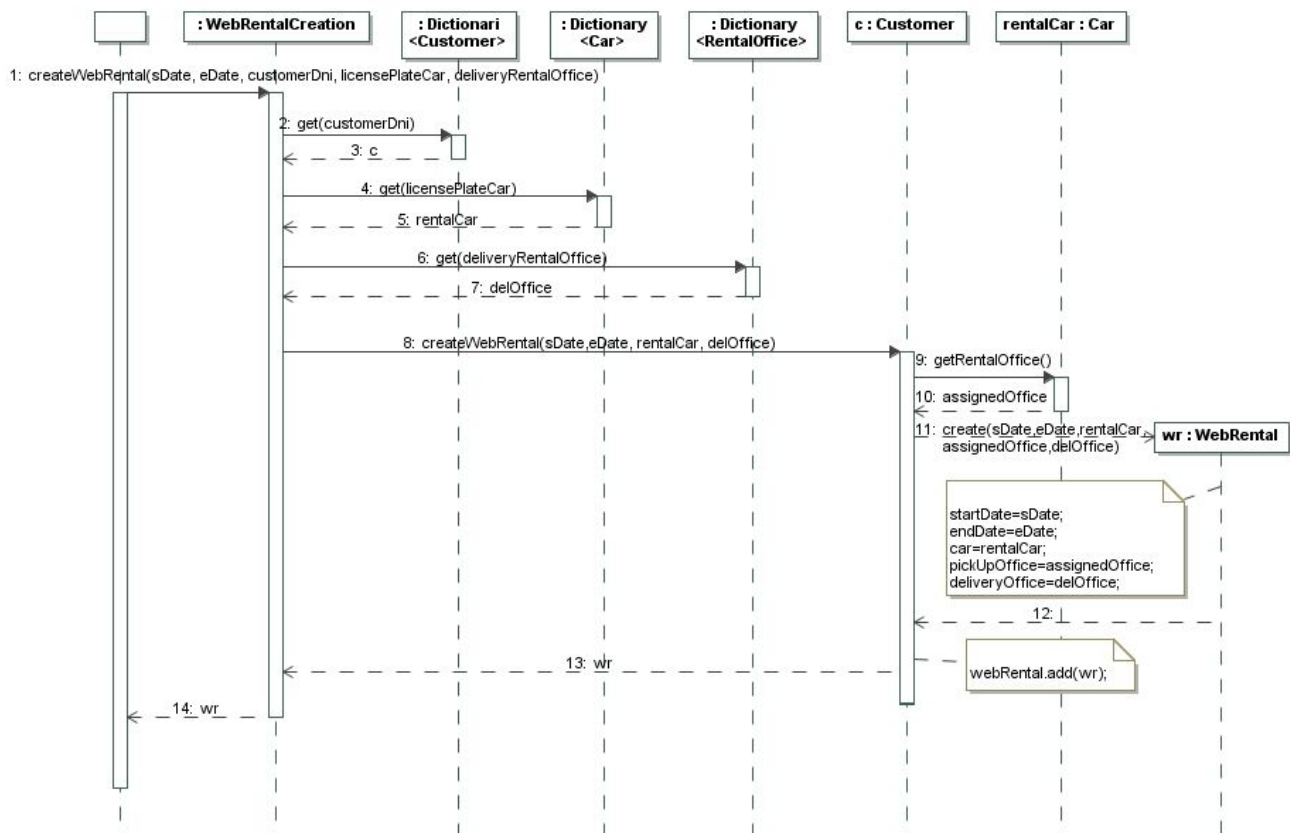
Nuestro sistema tiene que ser capaz de crear alquileres web para los clientes que lo deseen. Para crear estos alquileres es necesario disponer de la información del dni del cliente, la fecha inicial y final del alquiler, la matrícula del coche a alquilar y la oficina de entrega o devolución. Fijaos que la oficina de recogida es la misma que la oficina donde está asignado el coche. Cuando el cliente devuelva el coche se registrará la hora de la devolución. Se pide:

- a) ¿Qué patrones de diseño utilizarías para este caso?
- b) Explica cuál será la clase responsable de crear los alquileres y justifica tu respuesta (puede haber diferentes clases responsables pero solo tenéis que proporcionar una).
- c) Muestra el diagrama de secuencia o el pseudocódigo de la operación que crea un alquiler web, inicializa la fecha inicial y final del alquiler, asocia el cliente con el alquiler creado, y también el alquiler con el coche y con la oficina de devolución. Las precondiciones de la operación nos garantizan que: 1) el cliente no tiene otros alquileres que se solapen con el que queremos crear y 2) la fecha de inicio del alquiler es anterior a la fecha final y que es una fecha futura. Usad los diccionarios (módulo 3, secciones 5.2.2 y 5.2.3) para recuperar instancias a partir de su identificador.

Solución

- a) Utilizaremos el patrón *Creador*, ya que hay que crear instancias de la clase *WebRental*.
- b) Asignamos a la clase *Customer* la responsabilidad de crear un alquiler web puesto que esta clase registra los objetos alquileres. Podríamos también asignar esta responsabilidad al controlador *WebRentalCreation* puesto que tiene los datos de inicialización que se pasarán al alquiler web cuando este sea creado. También podemos asignar la responsabilidad a *Car* ya que también registra los objetos alquiler.
- c) El diagrama de secuencia muestra cómo el controlador obtiene de los diccionarios los objetos cliente, coche y oficina. El objeto cliente se utiliza para invocar a la operación *createWebRental* y el resto de objetos se pasan como parámetros a esta operación. Desde la operación *createWebRental* se invoca a la constructora de *WebRental*. Esta constructora

inicializa los atributos del objeto creado así como las asociaciones con el coche y con las oficinas.



Ejercicio 2 (10%)

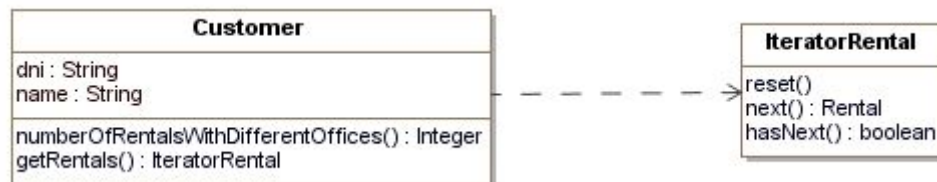
Supongamos ahora que queremos definir una operación *numberOfRentalsWithDifferentOffices()*: *Integer* en la clase *Customer* que devuelve el número de alquileres web que ha hecho un cliente *self* donde la oficina de recogida y de entrega es diferente. En este momento del diseño del sistema todavía no sabemos qué estructura de datos utilizaremos para guardar los alquileres que ha hecho/hace un cliente. Se pide:

- ¿Qué patrón de diseño utilizarías para diseñar esta operación? Justifica tu respuesta.
- Muestra la parte del diagrama de clases que se ve modificada como resultado de la aplicación del patrón utilizado.

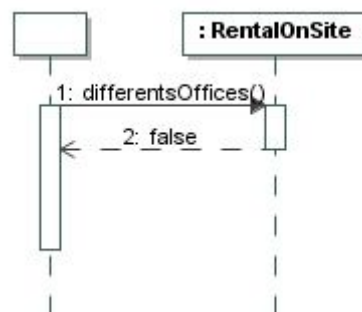
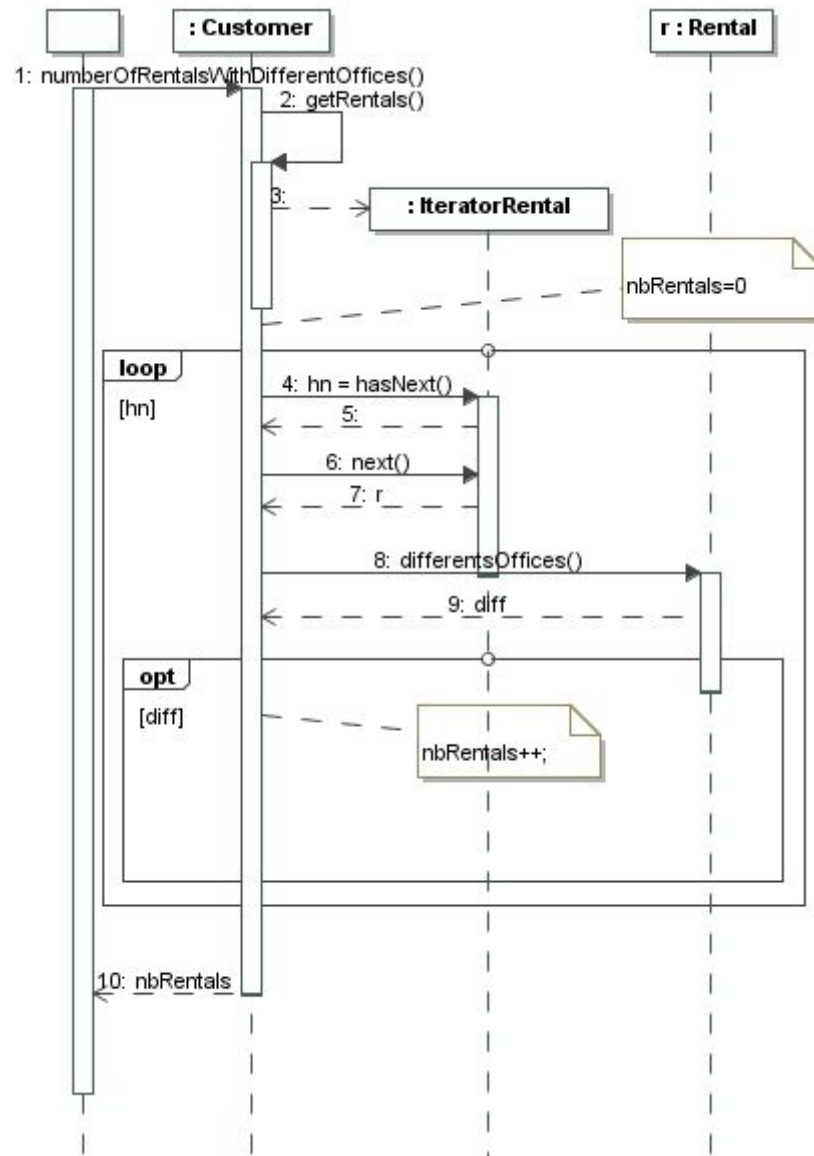
- c) Muestra el diagrama de secuencia o el pseudocódigo de la operación *numberOfRentalsWithDifferentOffices():Integer* de la clase *Customer*.

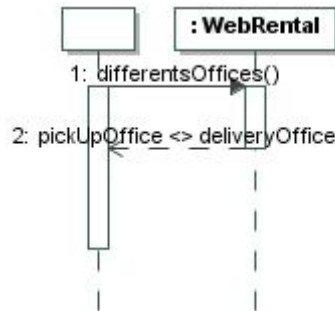
Solución

- a) Utilizaremos el patrón *Iterador* que permite recorrer los alquileres de un cliente encapsulando la estructura interna de esta información.
- b) El diagrama muestra la clase *Customer* y el iterador de los alquileres para recorrer los alquileres web del cliente.



- c) El diagrama de secuencia muestra el recorrido de los alquileres del cliente mediante el iterador. Por cada alquiler obtenido en el recorrido se pide si las oficinas de recogida y de entrega son diferentes. Para el caso de los alquileres en oficina será falso mientras que para los alquileres web se comprobará si coinciden las oficinas.





Ejercicio 3 (25%)

A menudo, los coches que la empresa de alquiler de coches pone a disposición de sus clientes se tienen que poner fuera de servicio (por reparaciones, para pasar la ITV, etc). Queremos representar esta situación en nuestro sistema para que cuando los coches estén fuera de servicio no puedan ser alquilados aunque registraremos, si hay, un coche que lo sustituye. Es por eso que nuestro sistema tendrá que proporcionar dos funcionalidades. La funcionalidad (1) poner un coche fuera de servicio (si ya está fuera de servicio o si está en servicio pero es sustituto de algún coche que está fuera de servicio, la funcionalidad no tendrá ningún efecto). Esta funcionalidad pondrá el coche fuera de servicio y registrará la fecha hasta la cual estará fuera de servicio y, si hay, buscará y registrará también un coche sustituto (será cualquier coche del mismo modelo del coche que se pone fuera de servicio que esté asignado a la misma oficina y que esté en servicio). La funcionalidad (2) pone un coche que estaba fuera de servicio en servicio.

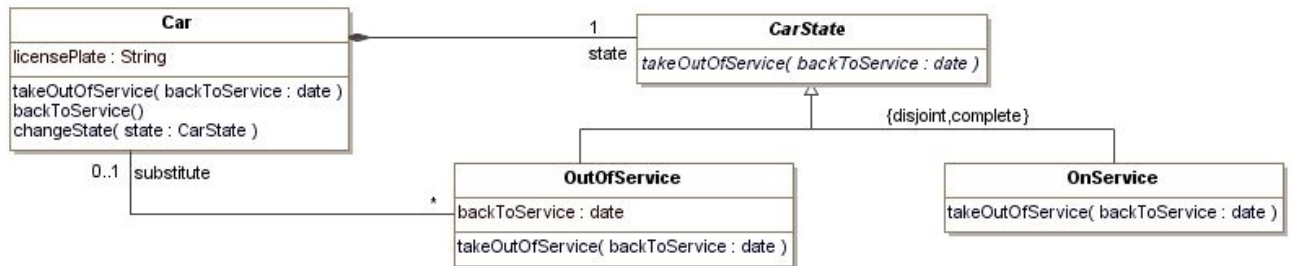
En concreto, nos centraremos en implementar la funcionalidad (1) añadiendo la operación *takeOutOfService(backToService:date)* de la clase *Car*. No hace falta que implementéis la funcionalidad (2).

- ¿Qué patrón de diseño recomendarías para representar la información descrita (si es que recomiendas alguno)?
- Muestra el diagrama de clases resultante de añadir estas funcionalidades.
- Muestra el diagrama de secuencia o el pseudocódigo de la operación *takeOutOfService* de la clase *Car*.

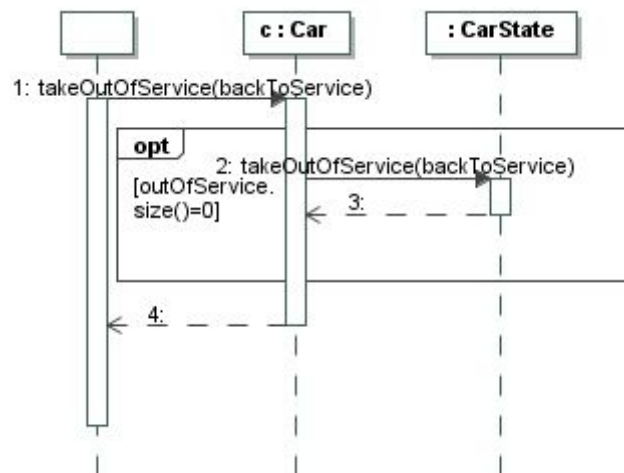
Solución

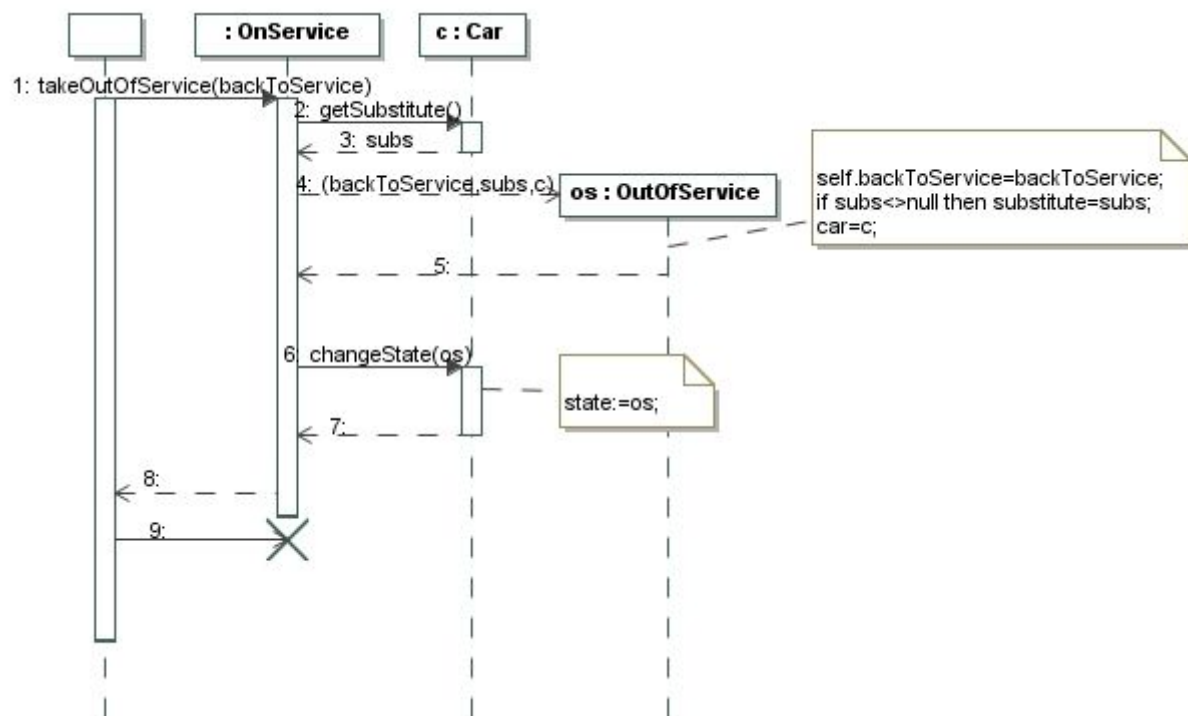
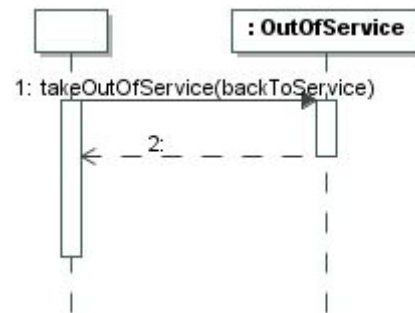
- Cómo dice el enunciado un coche puede estar en dos estados diferentes. Por lo tanto, el patrón que aplicaremos es el patrón *Estado*.

- b) El diagrama de clases muestra la aplicación del patrón *Estado* a la clase *Car*. Se han definido dos estados, el estado fuera de servicio y en servicio. Hace falta también añadir una nueva restricción de integridad: El sustituto de un coche fuera de servicio, si hay, tiene que ser un coche que esté en servicio.

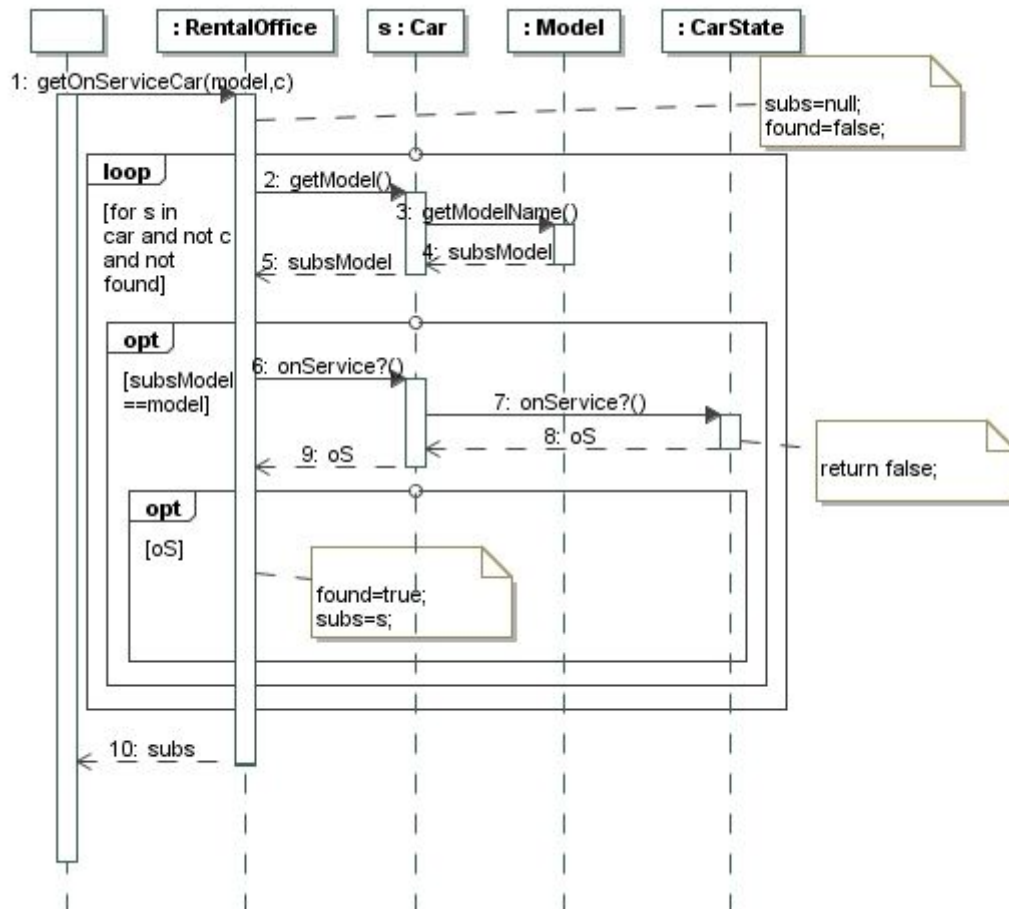
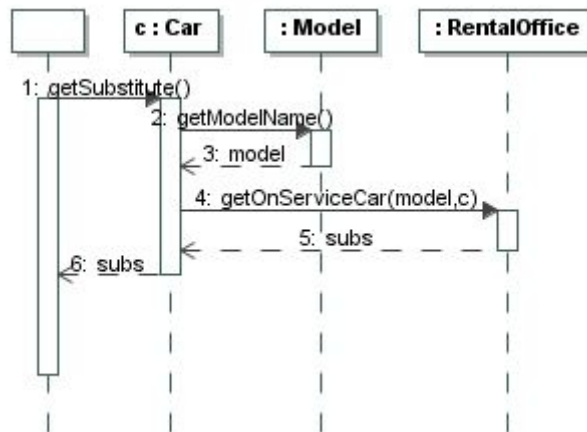


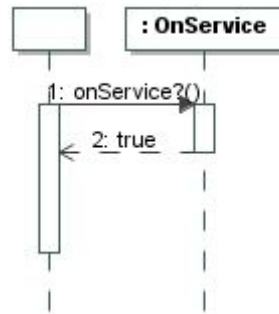
- c) El diagrama de secuencia muestra la definición de la operación *takeOutOfService* de la clase *Car*. Esta operación invoca a otra del mismo nombre al objeto *CarState* que está asociado con el coche. Esta última operación es abstracta y se define de forma diferente en sus subclases (estado *OutOfService* y *OnService*).





Nota: El diagrama de secuencia anterior añadirá navegabilidad de *CarState* a *Car*.





Ejercicio 4 (25%)

Cuando los clientes de la empresa de alquiler de coches alquilan un coche, el sistema que estamos construyendo tiene que proporcionarles el precio del alquiler. Este precio lo calcula la operación *getPrice():Integer* de la siguiente forma: 1) si es un alquiler de oficina o un alquiler web con la misma oficina de recogida y de entrega, el precio será el precio del modelo del vehículo por día (*pricePerDay*) * [*endDate* - *startDate*] y 2); si es un alquiler web con diferente oficina de recogida que de entrega, el precio será el precio del modelo del vehículo por día (*pricePerDay*) * [*endDate* - *startDate*] + *feeForDelivery*. Se pide:

- ¿Qué patrón de diseño es adecuado aplicar en esta situación?
- Haz el diagrama de clases con las operaciones resultantes de aplicar este patrón (podéis proporcionar solo la parte de aplicación del patrón).
- Haz el pseudocódigo o el diagrama de secuencia de la operación *getPrice():Integer* de la clase *Rental* y de todas las operaciones que sean invocadas desde esta operación.

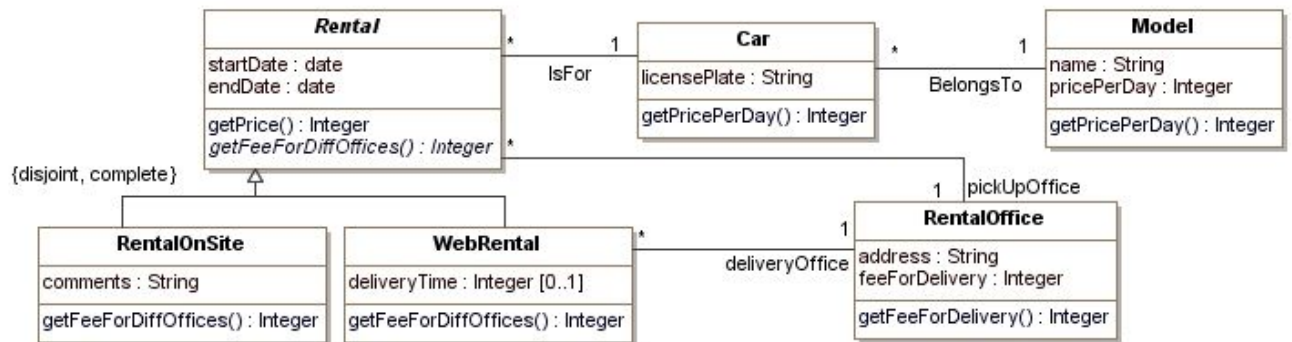
Nota:

- Se valorará especialmente que en la solución propuesta no haya repetición de código.

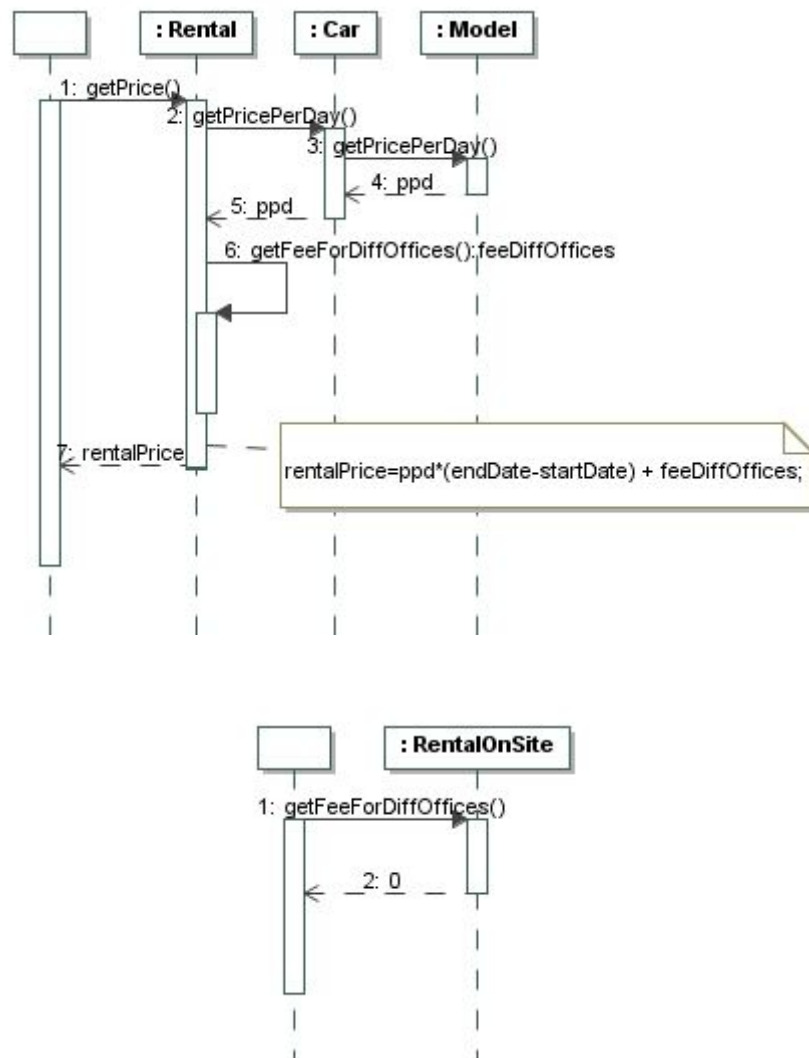
Solución

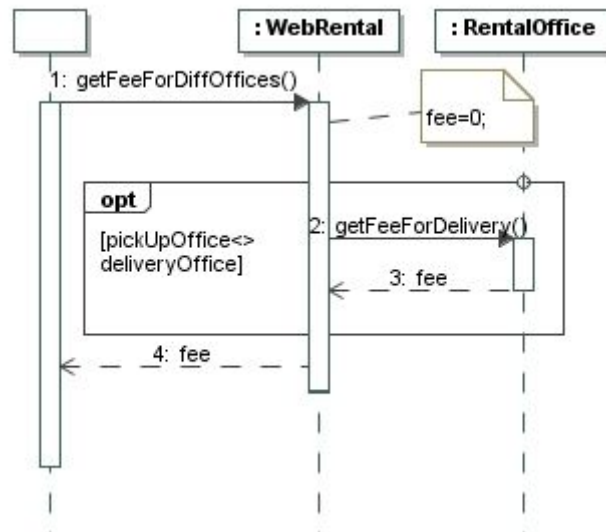
- Aplicaremos el patrón *método Plantilla* para diseñar la operación *getPrice()* que calculará el precio de un alquiler. La parte común corresponde al cálculo de (*pricePerDay*) * [*endDate* - *startDate*] y la parte específica a la obtención de *feeForDelivery* cuando el alquiler es web y tiene oficinas de recogida y entrega diferentes.

- b) El diagrama de clases resultante de la aplicación del patrón método plantilla a las operaciones *getPrice()* se encuentra a continuación.



- c) El diagrama de secuencia de la operación *getPrice():Integer* de la clase *Rental* se encuentra a continuación:





Una solución alternativa sería definir la operación `getFeeForDiffOffices():Integer` concreta en *Rental* y redefinirla en *WebRental*.

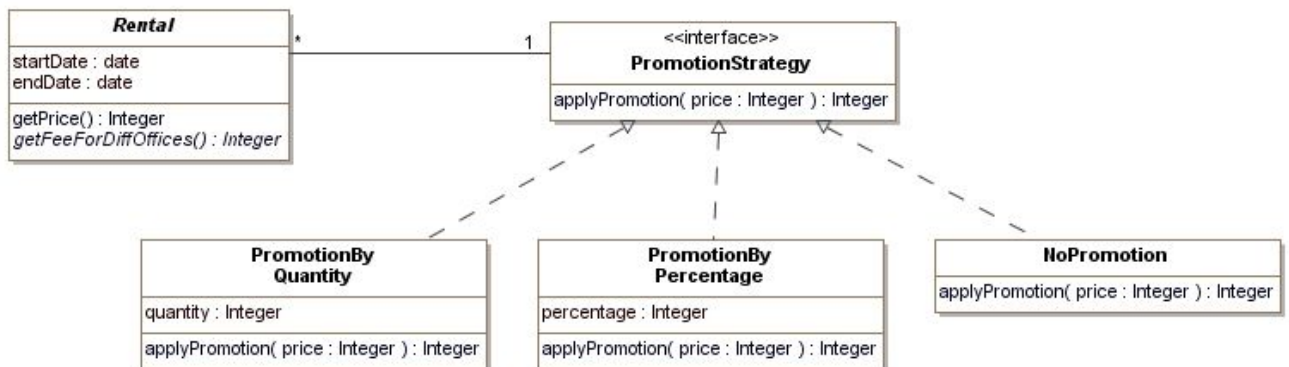
Ejercicio 5 (25%)

Después de diseñar la operación `getPrice():Integer` de la clase *Rental* en el ejercicio anterior, la empresa de alquiler de coches nos pide modificarla para añadir al cálculo de precios la posibilidad de hacer promociones que implican descuentos de estos precios. Inicialmente, la empresa ofrecerá dos tipos de promociones: por cantidad y por porcentaje. La promoción por cantidad permitirá decrementar el precio del alquiler en la cantidad indicada en la promoción. La promoción por porcentaje decrementará el precio del alquiler en el porcentaje indicado en la promoción. Las promociones se asignan a los alquileres en el momento de su creación. Evidentemente, es posible que a algunos alquileres no se les aplique ninguna promoción. Las promociones que se asignan a los alquileres son determinadas por una política de la empresa que no impacta al diseño de nuestra operación (impactará a la operación que crea los alquileres). Eso sí, la empresa quiere que mientras no se haga el pago del alquiler, si aparecen nuevas promociones, se apliquen a los alquileres siempre y cuando sean más favorables (no nos tenemos que preocupar tampoco de estos cambios, son gestionados por otras operaciones). Se pide:

- ¿Qué patrón de diseño recomendarías para este caso?
- Muestra el diagrama de clases resultante de la aplicación del patrón.
- Muestra el nuevo diagrama de secuencia o el pseudocódigo de la operación `getPrice():Integer` de la clase *Rental*.

Solución

- Aplicaremos el patrón *Estrategia*, ya que nos permite definir una familia de algoritmos (en nuestro caso las operaciones que permiten aplicar un descuento al precio aplicando la promoción asociada) y permite cambiar la promoción en tiempo de ejecución.
- El diagrama de clases muestra la aplicación del patrón estrategia para representar tres estrategias (*PromotionByQuantity*, *PromotionByPercentage* y *NoPromotion*) que permitirán aplicar el descuento por promoción.



- El diagrama de secuencia muestra la definición de la operación `getPrice():Integer` de la clase *Rental* (se han obviado las operaciones que no cambian respecto a la solución del ejercicio anterior). Esta operación invoca a otra de la interfaz *PromotionStrategy* que está asociada al alquiler. Esta última operación se define de forma diferente a las clases que implementan la interfaz.

